University Institute of Computing
Chandigarh University, Gharuan, Mohali

P R O J E C T R E P O R T

O N

Online Learning Management System (LMS)

Subject: - Database Management System
Subject Code: - 25CAT-203

SUBMITTED BY:

Name: Somya Gupta **UID: 25BCD10029**

Branch: UIC **Group: B, Section: BCD-3**

Semester: 03

SUBMITTED TO:

Name: Renu Jangra Ma'am **D.O. SUBMISSION: _____**

Designation: AP **Sign: _____**

INDEX

Sr. No	Content	Page No
01	Title Page	01
02	Index	02
03	Introduction	03
04	Aim	03
05	Objectives	03
06	Software and Tools Used	03
07	Database Design	04
08	ER Diagram	06
09	SQL Implementation and Query Explanations (Code with Explanation)	07-10
10	Output	11-21
11	Conclusion	22
12	Learning Outcomes	22
13	GitHub Link :- https://github.com/Somya-ds/Online-Learning-Management-System/tree/main	

Online Learning Management System (LMS)

I. Introduction

An Online Learning Management System (LMS) is a database-driven application designed to manage and organize online educational activities. It provides a structured way to store and retrieve information about students, instructors, departments, courses, learning modules, educational resources, assessments, submissions, and grades.

The main purpose of this project is to develop a relational database that simplifies the management of online learning activities. It maintains relationships between different entities and allows users to retrieve meaningful information through SQL queries.

The system uses **MySQL 8.0** to implement tables, constraints, relationships, joins, aggregate functions, subqueries, views, and indexes. These features help maintain data consistency and support efficient data retrieval.

II. Aim

To design and implement a relational database for an Online Learning Management System using MySQL 8.0 to manage educational information, course enrollments, learning resources, assessments, student submissions, and academic grades.

III. Objectives

1. To design a relational database consisting of ten interconnected tables.
2. To create tables using appropriate primary keys, foreign keys, and constraints.
3. To store and manage student, instructor, department, and course information.
4. To maintain student enrollment records and course learning materials.
5. To manage assessments, student submissions, and grades.
6. To retrieve related information using SQL joins and nested queries.
7. To perform data analysis using aggregate functions, GROUP BY, and HAVING.
8. To implement IN, EXISTS, ANY, and ALL operators.
9. To create views and indexes for convenient data access and searching.
10. To understand how relational databases support online educational platforms.

IV. Software and Tools Used

Table 1: Software and tools used in the project

Component	Details
Database Management System	MySQL 8.0 (MySQL Community Server – GPL, version 8.0.46)
Interface	MySQL 8.0 Command Line Client
Query Language	Structured Query Language (SQL)
Operating System	Windows
Database Model	Relational Database Model

V. Database Design

The database contains ten tables that represent the major components of the learning management system. Sample data was inserted into every table so that all queries could be tested on meaningful records.

Table 2: Tables of the LMS database, their purpose and number of records inserted

Table Name	Purpose	Records
Department	Stores department names and locations.	4
Student	Stores student details and department information.	8
Instructor	Stores instructor details and areas of specialization.	5
Course	Stores course descriptions, credits, instructors, and departments.	6
Enrollment	Records student enrollment in courses.	13
Module	Stores the modules or lessons belonging to courses.	12
Learning_Resource	Stores educational resources such as PDFs, videos, and articles.	12
Assessment	Stores quizzes, assignments, and examinations.	8
Submission	Records student submissions for assessments.	12
Grade	Stores marks, grades, and instructor feedback.	12

1. Primary Keys

Each table has a primary key that uniquely identifies its records, such as `Student_ID` in Student and `Course_ID` in Course.

2. Foreign Keys

Foreign keys connect related tables and help maintain referential integrity. For example, `Student.Department_ID` references `Department.Department_ID`, while `Course.Instructor_ID` references `Instructor.Instructor_ID`. In addition, UNIQUE constraints on (`Student_ID`, `Course_ID`) in Enrollment and (`Assessment_ID`, `Student_ID`) in Submission prevent duplicate enrollments and duplicate submissions, and the UNIQUE constraint on `Grade.Submission_ID` ensures that a submission has at most one grade.

Table 3: Structure of the tables (attributes, keys and constraints)

Table	Attributes	Keys and Constraints
Department	Department_ID, Department_Name, Location	PK: Department_ID Department_Name NOT NULL, UNIQUE
Student	Student_ID, Student_Name, Email, Phone, Department_ID, Join_Date	PK: Student_ID FK: Department_ID Email UNIQUE
Instructor	Instructor_ID, Instructor_Name, Email, Specialization, Department_ID	PK: Instructor_ID FK: Department_ID Email UNIQUE
Course	Course_ID, Course_Name, Course_Description, Credits, Instructor_ID, Department_ID	PK: Course_ID FK: Instructor_ID, Department_ID

Table	Attributes	Keys and Constraints
Enrollment	Enrollment_ID, Student_ID, Course_ID, Enrollment_Date, Status	PK: Enrollment_ID FK: Student_ID, Course_ID UNIQUE (Student_ID, Course_ID)
Module	Module_ID, Course_ID, Module_Name, Module_Description, Module_Order	PK: Module_ID FK: Course_ID
Learning_Resource	Resource_ID, Module_ID, Resource_Name, Resource_Type, Resource_URL	PK: Resource_ID FK: Module_ID
Assessment	Assessment_ID, Course_ID, Assessment_Name, Assessment_Type, Max_Marks, Assessment_Date	PK: Assessment_ID FK: Course_ID
Submission	Submission_ID, Assessment_ID, Student_ID, Submission_Date, Submission_Status	PK: Submission_ID FK: Assessment_ID, Student_ID UNIQUE (Assessment_ID, Student_ID)
Grade	Grade_ID, Submission_ID, Marks_Obtained, Grade, Feedback	PK: Grade_ID FK: Submission_ID (UNIQUE)

Important Relationships (from the ER Diagram)

A department can have multiple students, instructors, and courses, and an instructor can teach multiple courses. Students and courses have a many-to-many relationship that is implemented through the Enrollment table. A course can contain multiple modules and assessments, and a module can contain multiple learning resources. Students submit assessments through Submission, and each submission can have zero or one grade.

Table 4: Relationships between the tables

No.	Relationship	Cardinality	Foreign Key Used
1	Department → Student	1 : M	Student.Department_ID → Department.Department_ID
2	Department → Instructor	1 : M	Instructor.Department_ID → Department.Department_ID
3	Department → Course	1 : M	Course.Department_ID → Department.Department_ID
4	Instructor → Course	1 : M	Course.Instructor_ID → Instructor.Instructor_ID
5	Student → Enrollment	1 : M	Enrollment.Student_ID → Student.Student_ID
6	Course → Enrollment	1 : M	Enrollment.Course_ID → Course.Course_ID
7	Course → Module	1 : M	Module.Course_ID → Course.Course_ID
8	Module → Learning_Resource	1 : M	Learning_Resource.Module_ID → Module.Module_ID
9	Course → Assessment	1 : M	Assessment.Course_ID → Course.Course_ID
10	Student → Submission	1 : M	Submission.Student_ID → Student.Student_ID
11	Assessment → Submission	1 : M	Submission.Assessment_ID → Assessment.Assessment_ID
12	Submission → Grade	1 : 0..1	Grade.Submission_ID → Submission.Submission_ID (UNIQUE)

VI. ER Diagram

The Entity-Relationship (ER) diagram represents the entities, attributes, primary keys, foreign keys, and relationships in the database. It illustrates how departments, students, instructors, courses, enrollments, modules, learning resources, assessments, submissions, and grades are connected.

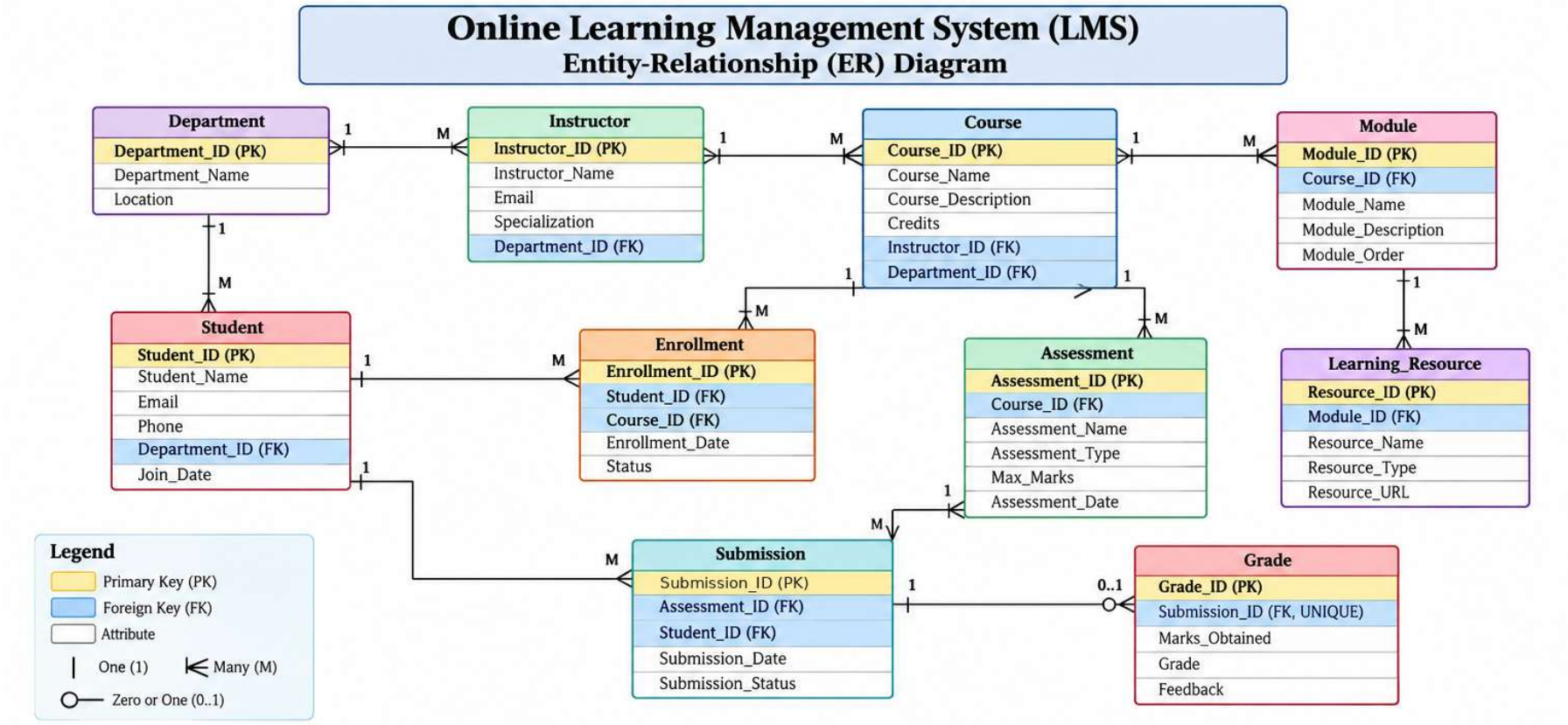


Figure 1: Entity-Relationship (ER) diagram of the Online Learning Management System

VII. SQL Implementation and Query Explanations

The database was implemented using the MySQL 8.0 Command Line Client. SQL commands were used to create the database, define tables, insert sample records, retrieve information, and implement advanced database operations. The complete script runs in sequence: the database and tables are created first, then data is inserted, and finally 26 queries are executed (basic queries 1 – 5, joins 6 – 10, aggregate functions 11 – 14, GROUP BY and HAVING 15 – 16, subqueries 17 – 18, IN 19 – 20, EXISTS 21 – 22, ANY and ALL 23 – 24, a view 25 and an index 26). The important statements are explained below.

1. Database and Table Creation

The `CREATE DATABASE` and `CREATE TABLE` statements establish the database structure and define the columns, data types, primary keys, foreign keys, and constraints. The `drop database if exists` statement makes the script re-runnable.

```
drop database if exists LMS;
create database LMS;
use LMS;

create table Department (
    Department_ID int primary key,
    Department_Name varchar(100) not null unique,
    Location varchar(100)
);
```

- `primary key` uniquely identifies each row; `not null` and `unique` protect important columns such as names and e-mail addresses.
- `foreign key ... references` links one table to another (for example `Student` to `Department`), so a row cannot refer to a record that does not exist.

Foreign keys and composite UNIQUE constraints are used in the Enrollment table, and the Grade table uses a UNIQUE foreign key to model the one-to-zero-or-one relationship with Submission:

```
create table Enrollment (
    Enrollment_ID int primary key,
    Student_ID int,
    Course_ID int,
    Enrollment_Date date,
    Status varchar(20),
    foreign key (Student_ID) references Student(Student_ID),
    foreign key (Course_ID) references Course(Course_ID),
    unique (Student_ID, Course_ID)
);

create table Grade (
    Grade_ID int primary key,
    Submission_ID int unique,
    Marks_Obtained decimal(5,2),
    Grade varchar(5),
    Feedback varchar(255),
    foreign key (Submission_ID) references Submission(Submission_ID)
);
```

2. Data Insertion

The `INSERT INTO` statement was used to insert multiple records into a table through a single statement, which made sample data entry more convenient.

```
insert into Department (Department_ID, Department_Name, Location)
values
(1, 'Computer Science', 'Block A'),
(2, 'Data Science', 'Block B'),
(3, 'Information Technology', 'Block C'),
(4, 'Artificial Intelligence', 'Block D');
```

3. Data Retrieval and Joins

The `SELECT` statement retrieves data from individual tables, while `JOIN` operations combine related information from multiple tables. For example, student names are displayed with their department names, and courses with their instructor names. The query below lists each student with the enrolled course and status by joining three tables.

```
select Student.Student_Name, Course.Course_Name, Enrollment.Status
from Enrollment
join Student
on Enrollment.Student_ID = Student.Student_ID
join Course
on Enrollment.Course_ID = Course.Course_ID;
```

4. Aggregate Functions, GROUP BY and HAVING

`COUNT()`, `AVG()`, `MAX()`, and `MIN()` calculate the total number of students, average marks, highest marks, and lowest marks. `GROUP BY` groups records by course or assessment, and `HAVING` filters the grouped results, for example to find courses with more than one enrollment.

```
select max(Marks_Obtained) as Highest_Marks,
       min(Marks_Obtained) as Lowest_Marks
from Grade;

select Course.Course_Name,
       count(Enrollment.Student_ID) as Total_Students
from Course
join Enrollment
on Course.Course_ID = Enrollment.Course_ID
group by Course.Course_ID, Course.Course_Name
having count(Enrollment.Student_ID) > 1;
```

5. Subqueries

Nested `SELECT` statements find courses with above-average credits and students who scored above the overall average marks. The inner query is evaluated first and its result is used by the outer query.

```
select Course_Name, Credits
from Course
where Credits > (
    select avg(Credits)
    from Course
);
```

6. IN and EXISTS

The `IN` operator identifies students belonging to a particular department and students who submitted assessments. The `EXISTS` operator finds instructors who teach at least one course, and `NOT EXISTS` finds students who have not submitted any assessment.

```
select Student_Name
from Student
where Department_ID in (
    select Department_ID
    from Department
    where Department_Name = 'Data Science'
);

select Instructor_Name
from Instructor i
where exists (
    select *
    from Course c
    where c.Instructor_ID = i.Instructor_ID
);
```

7. ANY and ALL

The `ANY` and `ALL` operators compare course credits against the credits of the courses offered by a particular department. `> ANY` returns course whose credits exceed at least one value returned by the subquery, whereas `> ALL` requires the credits to exceed every value.

```
select Course_Name, Credits
from Course
where Credits > any (
    select Credits
    from Course
    where Department_ID = 2
);
```

8. Views and Indexes

A view named `Student_Course_View` was created to display student enrollment details conveniently without rewriting the join each time. An index named `Idx_Course_Name` was created on `Course_Name` to support faster searches by course name.

```
create view Student_Course_View as
select Student.Student_ID, Student.Student_Name,
       Course.Course_Name, Enrollment.Enrollment_Date,
       Enrollment.Status
from Student
join Enrollment
on Student.Student_ID = Enrollment.Student_ID
join Course
on Enrollment.Course_ID = Course.Course_ID;

create index Idx_Course_Name
on Course(Course_Name);
```

Table 5: Summary of the 26 SQL queries executed

Query	Category	Description
1 – 2	Basic SELECT	Display all students and all courses
3 – 5	SELECT with WHERE	Students joined after 15 Jan 2026; 4-credit courses; assessments of at least 50 marks
6 – 10	JOIN	Students with departments; courses with instructors; students with courses; modules with courses; students with assessments and marks
11 – 14	Aggregate functions	Total students; average marks; highest and lowest marks; enrollments in each course
15 – 16	GROUP BY and HAVING	Courses with more than one enrollment; average marks of each assessment
17 – 18	Subqueries	Students scoring above the overall average; courses with above-average credits
19 – 20	IN	Students in Data Science department; students who submitted an assessment
21 – 22	EXISTS / NOT EXISTS	Instructors teaching at least one course; students with no submission
23 – 24	ANY / ALL	Courses with more credits than any / every Data Science course
25	View	Student_Course_View for student enrollment details
26	Index	Idx_Course_Name on Course(Course_Name)

VIII. Output

This section presents the screenshots taken in the MySQL 8.0 Command Line Client. They show the creation of the database and tables, the sample records, and the results of the 26 queries. Each output is followed by a short explanation of the results.

Output 1: Database Creation, Department, Student and Instructor Tables

- The client connected to **MySQL Community Server 8.0.46**; `create database LMS` and `use LMS` ran successfully.
- The **Department** table holds 4 departments (Computer Science, Data Science, Information Technology, Artificial Intelligence).
- The **Student** table holds 8 students and the **Instructor** table holds 5 instructors; every insert reported `Duplicates: 0  Warnings: 0`.

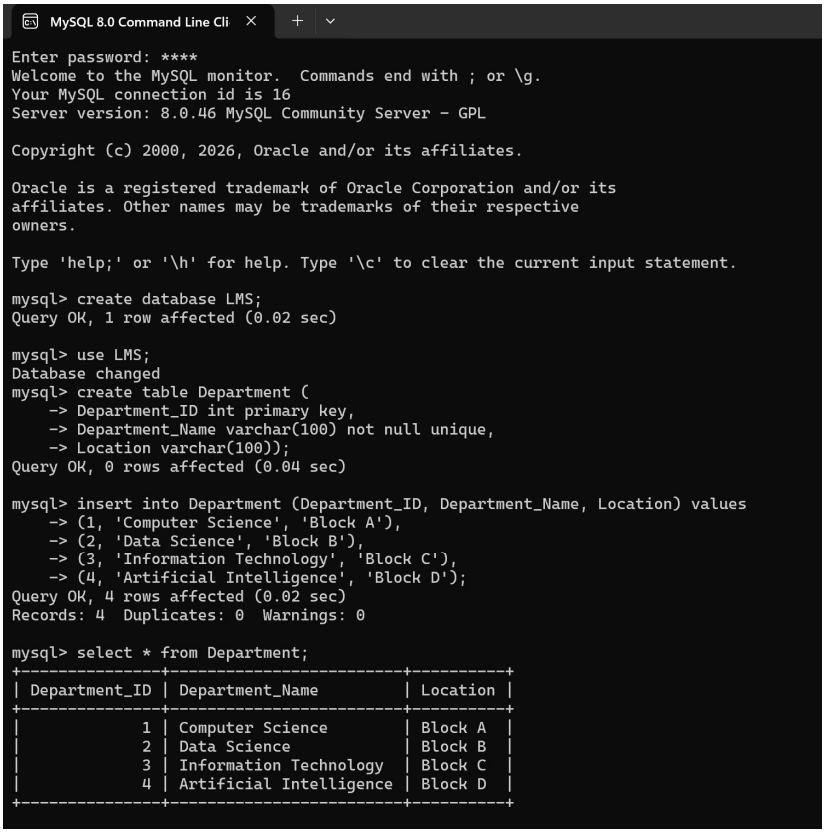


Figure 2: Connecting to MySQL 8.0, creating the LMS database, and creating and displaying the Department table

```
mysql> create table Student (
    -> Student_ID int primary key,
    -> Student_Name varchar(100) not null,
    -> Email varchar(100) unique,
    -> Phone varchar(15),
    -> Department_ID int,
    -> Join_Date date,
    -> foreign key (Department_ID) references Department(Department_ID));
Query OK, 0 rows affected (0.09 sec)

mysql> insert into Student (Student_ID, Student_Name, Email, Phone, Department_ID, Join_Date) values
    -> (101, 'Rahul Sharma', 'rahul@gmail.com', '9876543210', 1, '2026-01-10'),
    -> (102, 'Priya Singh', 'priya@gmail.com', '9876543211', 2, '2026-01-12'),
    -> (103, 'Aman Kumar', 'aman@gmail.com', '9876543212', 1, '2026-01-15'),
    -> (104, 'Neha Verma', 'neha@gmail.com', '9876543213', 3, '2026-01-18'),
    -> (105, 'Arjun Mehta', 'arjun@gmail.com', '9876543214', 4, '2026-01-20'),
    -> (106, 'Simran Kaur', 'simran@gmail.com', '9876543215', 2, '2026-01-22'),
    -> (107, 'Rohan Gupta', 'rohan@gmail.com', '9876543216', 1, '2026-01-25'),
    -> (108, 'Anjali Sharma', 'anjali@gmail.com', '9876543217', 3, '2026-01-27');
Query OK, 8 rows affected (0.02 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> select * from Student;
+------------+---------------+-----------------+------------+---------------+------------+
| Student_ID | Student_Name  | Email           | Phone      | Department_ID | Join_Date  |
+------------+---------------+-----------------+------------+---------------+------------+
|        101 | Rahul Sharma  | rahul@gmail.com | 9876543210 |             1 | 2026-01-10 |
|        102 | Priya Singh   | priya@gmail.com | 9876543211 |             2 | 2026-01-12 |
|        103 | Aman Kumar    | aman@gmail.com  | 9876543212 |             1 | 2026-01-15 |
|        104 | Neha Verma    | neha@gmail.com  | 9876543213 |             3 | 2026-01-18 |
|        105 | Arjun Mehta   | arjun@gmail.com | 9876543214 |             4 | 2026-01-20 |
|        106 | Simran Kaur   | simran@gmail.com| 9876543215 |             2 | 2026-01-22 |
|        107 | Rohan Gupta   | rohan@gmail.com | 9876543216 |             1 | 2026-01-25 |
|        108 | Anjali Sharma | anjali@gmail.com| 9876543217 |             3 | 2026-01-27 |
+------------+---------------+-----------------+------------+---------------+------------+
8 rows in set (0.00 sec)
```

Figure 3: Creating the Student table, inserting eight records, and displaying them

```
mysql> create table Instructor (
    -> Instructor_ID int primary key,
    -> Instructor_Name varchar(100) not null,
    -> Email varchar(100) unique,
    -> Specialization varchar(100),
    -> Department_ID int,
    -> foreign key (Department_ID) references Department(Department_ID));
Query OK, 0 rows affected (0.07 sec)

mysql> insert into Instructor (Instructor_ID, Instructor_Name, Email, Specialization, Department_ID) values
    -> (201, 'Dr. Rajesh Kumar', 'rajesh@lms.com', 'Database Management', 1),
    -> (202, 'Dr. Neha Sharma', 'neha@lms.com', 'Data Analytics', 2),
    -> (203, 'Prof. Amit Verma', 'amit@lms.com', 'Web Development', 3),
    -> (204, 'Dr. Priya Mehta', 'priya@lms.com', 'Machine Learning', 4),
    -> (205, 'Prof. Karan Singh', 'karan@lms.com', 'Python Programming', 1);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> select * from Instructor;
+---------------+-----------------+---------------+--------------------+---------------+
| Instructor_ID | Instructor_Name | Email         | Specialization     | Department_ID |
+---------------+-----------------+---------------+--------------------+---------------+
|           201 | Dr. Rajesh Kumar| rajesh@lms.com| Database Management|             1 |
|           202 | Dr. Neha Sharma | neha@lms.com  | Data Analytics     |             2 |
|           203 | Prof. Amit Verma| amit@lms.com  | Web Development    |             3 |
|           204 | Dr. Priya Mehta | priya@lms.com | Machine Learning   |             4 |
|           205 | Prof. Karan Singh| karan@lms.com| Python Programming |             1 |
+---------------+-----------------+---------------+--------------------+---------------+
5 rows in set (0.00 sec)
```

Figure 4: Creating the Instructor table, inserting five records, and displaying them

Output 2: Course and Enrollment Tables

- The **Course** table has 6 courses; Dr. Rajesh Kumar (201) teaches two of them.
- All **13 enrollment** records were inserted with one statement. Twelve are *Active* and one (Anjali Sharma in Python Programming) is *Completed*.

```
mysql> create table Course (
    -> Course_ID int primary key,
    -> Course_Name varchar(100) not null,
    -> Course_Description varchar(255),
    -> Credits int,
    -> Instructor_ID int,
    -> Department_ID int,
    -> foreign key (Instructor_ID) references Instructor(Instructor_ID),
    -> foreign key (Department_ID) references Department(Department_ID));
Query OK, 0 rows affected (0.09 sec)

mysql> insert into Course (Course_ID, Course_Name, Course_Description, Credits, Instructor_ID, Department_ID) values
    -> (301, 'Database Management Systems', 'Introduction to DBMS and SQL', 4, 201, 1),
    -> (302, 'Data Analytics', 'Data analysis and visualization', 4, 202, 2),
    -> (303, 'Web Development', 'HTML CSS and JavaScript', 3, 203, 3),
    -> (304, 'Machine Learning', 'Machine learning algorithms and applications', 4, 204, 4),
    -> (305, 'Python Programming', 'Python programming fundamentals', 3, 205, 1),
    -> (306, 'Advanced SQL', 'Advanced database queries and optimization', 3, 201, 1);
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> select * from Course;
+-----------+--------------------------+-----------------------------------------+---------+---------------+---------------+
| Course_ID | Course_Name              | Course_Description                      | Credits | Instructor_ID | Department_ID |
+-----------+--------------------------+-----------------------------------------+---------+---------------+---------------+
|       301 | Database Management Systems | Introduction to DBMS and SQL         |       4 |           201 |             1 |
|       302 | Data Analytics           | Data analysis and visualization         |       4 |           202 |             2 |
|       303 | Web Development          | HTML CSS and JavaScript                 |       3 |           203 |             3 |
|       304 | Machine Learning         | Machine learning algorithms and applications |  4 |           204 |             4 |
|       305 | Python Programming       | Python programming fundamentals         |       3 |           205 |             1 |
|       306 | Advanced SQL             | Advanced database queries and optimization |    3 |           201 |             1 |
+-----------+--------------------------+-----------------------------------------+---------+---------------+---------------+
6 rows in set (0.00 sec)
```

Figure 5: Creating the Course table with foreign keys to Instructor and Department, and displaying six courses

```
mysql> create table Enrollment (
    -> Enrollment_ID int primary key,
    -> Student_ID int,
    -> Course_ID int,
    -> Enrollment_Date date,
    -> Status varchar(20),
    -> foreign key (Student_ID) references Student(Student_ID),
    -> foreign key (Course_ID) references Course(Course_ID),
    -> unique (Student_ID, Course_ID));
Query OK, 0 rows affected (0.07 sec)

mysql> insert into Enrollment (Enrollment_ID, Student_ID, Course_ID, Enrollment_Date, Status) values
    -> (401, 101, 301, '2026-02-01', 'Active'),
    -> (402, 101, 305, '2026-02-02', 'Active'),
    -> (403, 102, 302, '2026-02-01', 'Active'),
    -> (404, 102, 304, '2026-02-03', 'Active'),
    -> (405, 103, 301, '2026-02-02', 'Active'),
    -> (406, 103, 306, '2026-02-04', 'Active'),
    -> (407, 104, 303, '2026-02-01', 'Active'),
    -> (408, 105, 304, '2026-02-05', 'Active'),
    -> (409, 106, 302, '2026-02-03', 'Active'),
    -> (410, 107, 305, '2026-02-04', 'Active'),
    -> (411, 108, 303, '2026-02-05', 'Active'),
    -> (412, 107, 301, '2026-02-06', 'Active'),
    -> (413, 108, 305, '2026-02-07', 'Completed');
Query OK, 13 rows affected (0.02 sec)
Records: 13  Duplicates: 0  Warnings: 0
```

Figure 6: Creating the Enrollment table with a unique (Student_ID, Course_ID) pair and inserting 13 records

```
mysql> select * from Enrollment;
+---------------+------------+-----------+-----------------+-----------+
| Enrollment_ID | Student_ID | Course_ID | Enrollment_Date | Status    |
+---------------+------------+-----------+-----------------+-----------+
|           401 |        101 |       301 | 2026-02-01      | Active    |
|           402 |        101 |       305 | 2026-02-02      | Active    |
|           403 |        102 |       302 | 2026-02-01      | Active    |
|           404 |        102 |       304 | 2026-02-03      | Active    |
|           405 |        103 |       301 | 2026-02-02      | Active    |
|           406 |        103 |       306 | 2026-02-04      | Active    |
|           407 |        104 |       303 | 2026-02-01      | Active    |
|           408 |        105 |       304 | 2026-02-05      | Active    |
|           409 |        106 |       302 | 2026-02-03      | Active    |
|           410 |        107 |       305 | 2026-02-04      | Active    |
|           411 |        108 |       303 | 2026-02-05      | Active    |
|           412 |        107 |       301 | 2026-02-06      | Active    |
|           413 |        108 |       305 | 2026-02-07      | Completed |
+---------------+------------+-----------+-----------------+-----------+
13 rows in set (0.00 sec)

mysql> create table Module (
    -> Module_ID int primary key,
    -> Course_ID int,
    -> Module_Name varchar(100) not null,
    -> Module_Description varchar(255),
    -> Module_Order int,
    -> foreign key (Course_ID) references Course(Course_ID));
Query OK, 0 rows affected (0.06 sec)

mysql> insert into Module (Module_ID, Course_ID, Module_Name, Module_Description, Module_Order) values
    -> (501, 301, 'Introduction to DBMS', 'Basic database concepts', 1),
    -> (502, 301, 'SQL Fundamentals', 'Introduction to SQL commands', 2),
    -> (503, 302, 'Introduction to Data Analytics', 'Basics of data analysis', 1),
    -> (504, 302, 'Data Visualization', 'Charts and visualization techniques', 2),
    -> (505, 303, 'HTML Basics', 'Introduction to HTML', 1),
    -> (506, 303, 'CSS Fundamentals', 'Styling web pages using CSS', 2),
    -> (507, 304, 'Introduction to Machine Learning', 'Basic machine learning concepts', 1),
    -> (508, 304, 'Supervised Learning', 'Classification and regression', 2),
    -> (509, 305, 'Python Basics', 'Variables and data types', 1),
    -> (510, 305, 'Control Statements', 'Loops and conditional statements', 2),
```

Figure 7: Enrollment table contents, and the start of the Module table creation and data insertion

Output 3: Module, Learning Resource, Assessment, Submission and Grade Tables

- The **Module** table has 12 modules (two per course), and **Learning_Resource** has 12 resources of type PDF, Video or Article.
- The **Assessment** table has 8 assessments (Quiz, Assignment and Exam) and **Submission** has 12 records, each unique per student and assessment.
- **Grade** has 12 records, one for each submission. `SHOW TABLES` lists all **ten tables**, confirming that the schema was created correctly.

```
    -> (511, 306, 'Advanced Joins', 'Complex SQL joins', 1),
    -> (512, 306, 'Query Optimization', 'Improving SQL query performance', 2);
Query OK, 12 rows affected (0.01 sec)
Records: 12  Duplicates: 0  Warnings: 0

mysql> select * from Module;
+-----------+-----------+--------------------------------+-----------------------------------+--------------+
| Module_ID | Course_ID | Module_Name                    | Module_Description                | Module_Order |
+-----------+-----------+--------------------------------+-----------------------------------+--------------+
|       501 |       301 | Introduction to DBMS           | Basic database concepts           |            1 |
|       502 |       301 | SQL Fundamentals               | Introduction to SQL commands      |            2 |
|       503 |       302 | Introduction to Data Analytics | Basics of data analysis           |            1 |
|       504 |       302 | Data Visualization             | Charts and visualization techniques |            2 |
|       505 |       303 | HTML Basics                    | Introduction to HTML              |            1 |
|       506 |       303 | CSS Fundamentals               | Styling web pages using CSS       |            2 |
|       507 |       304 | Introduction to Machine Learning | Basic machine learning concepts |            1 |
|       508 |       304 | Supervised Learning            | Classification and regression     |            2 |
|       509 |       305 | Python Basics                  | Variables and data types          |            1 |
|       510 |       305 | Control Statements             | Loops and conditional statements  |            2 |
|       511 |       306 | Advanced Joins                 | Complex SQL joins                 |            1 |
|       512 |       306 | Query Optimization             | Improving SQL query performance   |            2 |
+-----------+-----------+--------------------------------+-----------------------------------+--------------+
12 rows in set (0.00 sec)

mysql> create table Learning_Resource (
    -> Resource_ID int primary key,
    -> Module_ID int,
    -> Resource_Name varchar(150) not null,
    -> Resource_Type varchar(30),
    -> Resource_URL varchar(255),
    -> foreign key (Module_ID) references Module(Module_ID)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

Figure 8: Remaining Module records, Module table contents, and creation of the Learning_Resource table

```
mysql> insert into Learning_Resource (Resource_ID, Module_ID, Resource_Name, Resource_Type, Resource_URL) values
    -> (601, 501, 'DBMS Introduction Notes', 'PDF', 'https://example.com/dbms-notes'),
    -> (602, 502, 'SQL Basics Tutorial', 'Video', 'https://example.com/sql-basics'),
    -> (603, 503, 'Data Analytics Guide', 'PDF', 'https://example.com/analytics-guide'),
    -> (604, 504, 'Data Visualization Tutorial', 'Video', 'https://example.com/visualization'),
    -> (605, 505, 'HTML Learning Guide', 'PDF', 'https://example.com/html-guide'),
    -> (606, 506, 'CSS Tutorial', 'Video', 'https://example.com/css-tutorial'),
    -> (607, 507, 'Machine Learning Introduction', 'Article', 'https://example.com/ml-intro'),
    -> (608, 508, 'Supervised Learning Notes', 'PDF', 'https://example.com/supervised-learning'),
    -> (609, 509, 'Python Basics Tutorial', 'Video', 'https://example.com/python-basics'),
    -> (610, 510, 'Control Statements Guide', 'PDF', 'https://example.com/control-statements'),
    -> (611, 511, 'Advanced Joins Tutorial', 'Video', 'https://example.com/advanced-joins'),
    -> (612, 512, 'Query Optimization Guide', 'PDF', 'https://example.com/query-optimization');
Query OK, 12 rows affected (0.02 sec)
Records: 12  Duplicates: 0  Warnings: 0

mysql> create table Assessment (
    -> Assessment_ID int primary key,
    -> Course_ID int,
    -> Assessment_Name varchar(100) not null,
    -> Assessment_Type varchar(30),
    -> Max_Marks int,
    -> Assessment_Date date,
    -> foreign key (Course_ID) references Course(Course_ID));
Query OK, 0 rows affected (0.07 sec)

mysql> insert into Assessment (Assessment_ID, Course_ID, Assessment_Name, Assessment_Type, Max_Marks, Assessment_Date) values
    -> (701, 301, 'DBMS Quiz', 'Quiz', 20, '2026-03-01'),
    -> (702, 301, 'SQL Assignment', 'Assignment', 50, '2026-03-05'),
    -> (703, 302, 'Analytics Quiz', 'Quiz', 20, '2026-03-02'),
    -> (704, 302, 'Data Analysis Project', 'Assignment', 50, '2026-03-08'),
    -> (705, 303, 'HTML CSS Test', 'Quiz', 25, '2026-03-03'),
    -> (706, 304, 'ML Midterm Exam', 'Exam', 100, '2026-03-10'),
    -> (707, 305, 'Python Programming Assignment', 'Assignment', 50, '2026-03-06'),
    -> (708, 306, 'Advanced SQL Test', 'Exam', 100, '2026-03-12');
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

Figure 9: Inserting learning resources, and creating and populating the Assessment table

```
mysql> create table Submission (
    -> Submission_ID int primary key,
    -> Assessment_ID int,
    -> Student_ID int,
    -> Submission_Date date,
    -> Submission_Status varchar(20),
    -> foreign key (Assessment_ID) references Assessment(Assessment_ID),
    -> foreign key (Student_ID) references Student(Student_ID),
    -> unique (Assessment_ID, Student_ID));
Query OK, 0 rows affected (0.04 sec)

mysql> insert into Submission (Submission_ID, Assessment_ID, Student_ID, Submission_Date, Submission_Status) values
    -> (801, 701, 101, '2026-03-01', 'Submitted'),
    -> (802, 702, 101, '2026-03-05', 'Submitted'),
    -> (803, 701, 103, '2026-03-01', 'Submitted'),
    -> (804, 703, 102, '2026-03-02', 'Submitted'),
    -> (805, 704, 106, '2026-03-08', 'Submitted'),
    -> (806, 705, 104, '2026-03-03', 'Submitted'),
    -> (807, 706, 105, '2026-03-10', 'Submitted'),
    -> (808, 707, 107, '2026-03-06', 'Submitted'),
    -> (809, 707, 108, '2026-03-06', 'Submitted'),
    -> (810, 708, 103, '2026-03-12', 'Submitted'),
    -> (811, 703, 106, '2026-03-02', 'Submitted'),
    -> (812, 706, 102, '2026-03-10', 'Submitted');
Query OK, 12 rows affected (0.01 sec)
Records: 12  Duplicates: 0  Warnings: 0

mysql> create table Grade (
    -> Grade_ID int primary key,
    -> Submission_ID int unique,
    -> Marks_Obtained decimal(5,2),
    -> Grade varchar(5),
    -> Feedback varchar(255),
    -> foreign key (Submission_ID) references Submission(Submission_ID));
Query OK, 0 rows affected (0.07 sec)
```

Figure 10: Creating and populating the Submission table, and creating the Grade table

```
mysql> insert into Grade (Grade_ID, Submission_ID, Marks_Obtained, Grade, Feedback)
    -> values
    -> (901, 801, 18, 'A', 'Excellent performance'),
    -> (902, 802, 44, 'A', 'Very good SQL skills'),
    -> (903, 803, 15, 'B', 'Good understanding'),
    -> (904, 804, 17, 'A', 'Well done'),
    -> (905, 805, 42, 'A', 'Good analytical work'),
    -> (906, 806, 21, 'B', 'Good effort'),
    -> (907, 807, 85, 'A', 'Excellent understanding'),
    -> (908, 808, 46, 'A', 'Very good Python skills'),
    -> (909, 809, 39, 'B', 'Good work'),
    -> (910, 810, 88, 'A', 'Excellent SQL knowledge'),
    -> (911, 811, 14, 'B', 'Satisfactory performance'),
    -> (912, 812, 76, 'B', 'Good understanding');
Query OK, 12 rows affected (0.02 sec)
Records: 12  Duplicates: 0  Warnings: 0

mysql> show tables;
+------------------+
| Tables_in_lms    |
+------------------+
| assessment       |
| course           |
| department       |
| enrollment       |
| grade            |
| instructor       |
| learning_resource |
| module           |
| student          |
| submission       |
+------------------+
10 rows in set (0.03 sec)
```

Figure 11: Inserting the grade records and listing all ten tables with SHOW TABLES

Output 4: Verification of Table Contents

- All 12 submissions have the status *Submitted*, and the 12 grades range from 14 to 88 marks with grades A or B and written feedback.

```
mysql> select * from Learning_Resource;
+-------------+-----------+------------------------------+---------------+-----------------------------------------+
| Resource_ID | Module_ID | Resource_Name                | Resource_Type | Resource_URL                            |
+-------------+-----------+------------------------------+---------------+-----------------------------------------+
|         601 |       501 | DBMS Introduction Notes      | PDF           | https://example.com/dbms-notes          |
|         602 |       502 | SQL Basics Tutorial          | Video         | https://example.com/sql-basics          |
|         603 |       503 | Data Analytics Guide         | PDF           | https://example.com/analytics-guide     |
|         604 |       504 | Data Visualization Tutorial  | Video         | https://example.com/visualization       |
|         605 |       505 | HTML Learning Guide          | PDF           | https://example.com/html-guide          |
|         606 |       506 | CSS Tutorial                 | Video         | https://example.com/css-tutorial        |
|         607 |       507 | Machine Learning Introduction | Article       | https://example.com/ml-intro            |
|         608 |       508 | Supervised Learning Notes    | PDF           | https://example.com/supervised-learning |
|         609 |       509 | Python Basics Tutorial       | Video         | https://example.com/python-basics       |
|         610 |       510 | Control Statements Guide     | PDF           | https://example.com/control-statements  |
|         611 |       511 | Advanced Joins Tutorial      | Video         | https://example.com/advanced-joins      |
|         612 |       512 | Query Optimization Guide     | PDF           | https://example.com/query-optimization  |
+-------------+-----------+------------------------------+---------------+-----------------------------------------+
12 rows in set (0.00 sec)

mysql> select * from Assessment;
+---------------+-----------+-------------------------------+-----------------+-----------+-----------------+
| Assessment_ID | Course_ID | Assessment_Name               | Assessment_Type | Max_Marks | Assessment_Date |
+---------------+-----------+-------------------------------+-----------------+-----------+-----------------+
|           701 |       301 | DBMS Quiz                     | Quiz            |        20 | 2026-03-01      |
|           702 |       301 | SQL Assignment                | Assignment      |        50 | 2026-03-05      |
|           703 |       302 | Analytics Quiz                | Quiz            |        20 | 2026-03-02      |
|           704 |       302 | Data Analysis Project         | Assignment      |        50 | 2026-03-08      |
|           705 |       303 | HTML CSS Test                 | Quiz            |        25 | 2026-03-03      |
|           706 |       304 | ML Midterm Exam               | Exam            |       100 | 2026-03-10      |
|           707 |       305 | Python Programming Assignment | Assignment      |        50 | 2026-03-06      |
|           708 |       306 | Advanced SQL Test             | Exam            |       100 | 2026-03-12      |
+---------------+-----------+-------------------------------+-----------------+-----------+-----------------+
8 rows in set (0.00 sec)
```

Figure 12: Contents of the Learning_Resource and Assessment tables

```
mysql> select * from Submission;
+---------------+---------------+------------+-----------------+-------------------+
| Submission_ID | Assessment_ID | Student_ID | Submission_Date | Submission_Status |
+---------------+---------------+------------+-----------------+-------------------+
|           801 |           701 |        101 | 2026-03-01      | Submitted         |
|           802 |           702 |        101 | 2026-03-05      | Submitted         |
|           803 |           701 |        103 | 2026-03-01      | Submitted         |
|           804 |           703 |        102 | 2026-03-02      | Submitted         |
|           805 |           704 |        106 | 2026-03-08      | Submitted         |
|           806 |           705 |        104 | 2026-03-03      | Submitted         |
|           807 |           706 |        105 | 2026-03-10      | Submitted         |
|           808 |           707 |        107 | 2026-03-06      | Submitted         |
|           809 |           707 |        108 | 2026-03-06      | Submitted         |
|           810 |           708 |        103 | 2026-03-12      | Submitted         |
|           811 |           703 |        106 | 2026-03-02      | Submitted         |
|           812 |           706 |        102 | 2026-03-10      | Submitted         |
+---------------+---------------+------------+-----------------+-------------------+
12 rows in set (0.00 sec)

mysql> select * from Grade;
+----------+---------------+-----------------+-------+--------------------------+
| Grade_ID | Submission_ID | Marks_Obtained | Grade | Feedback                 |
+----------+---------------+-----------------+-------+--------------------------+
|      901 |           801 |          18.00 | A     | Excellent performance    |
|      902 |           802 |          44.00 | A     | Very good SQL skills     |
|      903 |           803 |          15.00 | B     | Good understanding       |
|      904 |           804 |          17.00 | A     | Well done                |
|      905 |           805 |          42.00 | A     | Good analytical work     |
|      906 |           806 |          21.00 | B     | Good effort              |
|      907 |           807 |          85.00 | A     | Excellent understanding  |
|      908 |           808 |          46.00 | A     | Very good Python skills  |
|      909 |           809 |          39.00 | B     | Good work                |
|      910 |           810 |          88.00 | A     | Excellent SQL knowledge  |
|      911 |           811 |          14.00 | B     | Satisfactory performance |
|      912 |           812 |          76.00 | B     | Good understanding       |
+----------+---------------+-----------------+-------+--------------------------+
12 rows in set (0.00 sec)
```

Figure 13: Contents of the Submission and Grade tables

Output 5: Basic SQL Queries (Queries 1 – 5)

- **Query 3** returned 5 students: Neha Verma, Arjun Mehta, Simran Kaur, Rohan Gupta and Anjali Sharma. Aman Kumar joined exactly on 15 January, so he is excluded by the > condition.
- **Query 4** returned three 4-credit courses: Database Management Systems, Data Analytics and Machine Learning.
- **Query 5** returned 5 assessments with 50 or more marks: the three assignments and the two exams.

```
mysql> select * from Student;
+------------+--------------+-----------------+------------+---------------+------------+
| Student_ID | Student_Name | Email           | Phone      | Department_ID | Join_Date  |
+------------+--------------+-----------------+------------+---------------+------------+
|        101 | Rahul Sharma | rahul@gmail.com | 9876543210 |             1 | 2026-01-10 |
|        102 | Priya Singh  | priya@gmail.com | 9876543211 |             2 | 2026-01-12 |
|        103 | Aman Kumar   | aman@gmail.com  | 9876543212 |             1 | 2026-01-15 |
|        104 | Neha Verma   | neha@gmail.com  | 9876543213 |             3 | 2026-01-18 |
|        105 | Arjun Mehta  | arjun@gmail.com | 9876543214 |             4 | 2026-01-20 |
|        106 | Simran Kaur  | simran@gmail.com| 9876543215 |             2 | 2026-01-22 |
|        107 | Rohan Gupta  | rohan@gmail.com | 9876543216 |             1 | 2026-01-25 |
|        108 | Anjali Sharma| anjali@gmail.com| 9876543217 |             3 | 2026-01-27 |
+------------+--------------+-----------------+------------+---------------+------------+
8 rows in set (0.00 sec)

mysql> select * from Course;
+-----------+---------------------------+-----------------------------------------+---------+---------------+---------------+
| Course_ID | Course_Name               | Course_Description                      | Credits | Instructor_ID | Department_ID |
+-----------+---------------------------+-----------------------------------------+---------+---------------+---------------+
|       301 | Database Management Systems | Introduction to DBMS and SQL          |       4 |           201 |             1 |
|       302 | Data Analytics            | Data analysis and visualization         |       4 |           202 |             2 |
|       303 | Web Development           | HTML CSS and JavaScript                 |       3 |           203 |             3 |
|       304 | Machine Learning          | Machine learning algorithms and applications |  4 |           204 |             4 |
|       305 | Python Programming        | Python programming fundamentals         |       3 |           205 |             1 |
|       306 | Advanced SQL              | Advanced database queries and optimization |    3 |           201 |             1 |
+-----------+---------------------------+-----------------------------------------+---------+---------------+---------------+
6 rows in set (0.00 sec)

mysql> select Student_ID, Student_Name, Join_Date
    -> from Student
    -> where Join_Date > '2026-01-15';
+------------+--------------+------------+
| Student_ID | Student_Name | Join_Date  |
+------------+--------------+------------+
|        104 | Neha Verma   | 2026-01-18 |
|        105 | Arjun Mehta  | 2026-01-20 |
|        106 | Simran Kaur  | 2026-01-22 |
|        107 | Rohan Gupta  | 2026-01-25 |
|        108 | Anjali Sharma| 2026-01-27 |
+------------+--------------+------------+
```

Figure 14: Queries 1 – 3: all students, all courses, and students who joined after 15 January 2026

```
mysql> select Course_Name, Credits
    -> from Course
    -> where Credits = 4;
+-----------------------------+---------+
| Course_Name                 | Credits |
+-----------------------------+---------+
| Database Management Systems |       4 |
| Data Analytics              |       4 |
| Machine Learning            |       4 |
+-----------------------------+---------+
3 rows in set (0.00 sec)

mysql> select Assessment_Name, Assessment_Type, Max_Marks
    -> from Assessment
    -> where Max_Marks >= 50;
+-------------------------------+-----------------+-----------+
| Assessment_Name               | Assessment_Type | Max_Marks |
+-------------------------------+-----------------+-----------+
| SQL Assignment                | Assignment      |        50 |
| Data Analysis Project         | Assignment      |        50 |
| ML Midterm Exam               | Exam            |       100 |
| Python Programming Assignment | Assignment      |        50 |
| Advanced SQL Test             | Exam            |       100 |
+-------------------------------+-----------------+-----------+
5 rows in set (0.00 sec)
```

Figure 15: Queries 4 – 5: 4-credit courses and assessments worth at least 50 marks

Output 6: Join Queries and Aggregate Functions (Queries 6 – 15)

- **Query 6** listed all 8 students with their department, and **Query 7** listed the 6 courses with their instructors.
- **Query 8** produced 13 rows (one per enrollment) and **Query 9** produced 12 rows (one per module).
- **Query 10** joined four tables (Grade, Submission, Student, Assessment) and returned the 12 marks with the student and assessment names.
- **Query 11** counted 8 students and **Query 12** gave an average of 42.083333 marks. **Query 13** returned a highest mark of 88.00 and a lowest mark of 14.00.
- **Query 14** (`LEFT JOIN`) lists every course with its enrollment count: Database Management Systems 3, Python Programming 3, Data Analytics 2, Web Development 2, Machine Learning 2 and Advanced SQL 1.
- **Query 15** (`HAVING count > 1`) returned 5 courses; Advanced SQL is excluded because it has only one enrollment.

```
mysql> select Student.Student_Name, Department.Department_Name
    -> from Student
    -> join Department
    -> on Student.Department_ID = Department.Department_ID;
+---------------+--------------------------+
| Student_Name  | Department_Name          |
+---------------+--------------------------+
| Arjun Mehta   | Artificial Intelligence  |
| Rahul Sharma  | Computer Science         |
| Aman Kumar    | Computer Science         |
| Rohan Gupta   | Computer Science         |
| Priya Singh   | Data Science             |
| Simran Kaur   | Data Science             |
| Neha Verma    | Information Technology   |
| Anjali Sharma | Information Technology   |
+---------------+--------------------------+
8 rows in set (0.00 sec)

mysql> select Course.Course_Name, Instructor.Instructor_Name
    -> from Course
    -> join Instructor
    -> on Course.Instructor_ID = Instructor.Instructor_ID;
+------------------------------+-------------------+
| Course_Name                  | Instructor_Name   |
+------------------------------+-------------------+
| Database Management Systems  | Dr. Rajesh Kumar  |
| Advanced SQL                 | Dr. Rajesh Kumar  |
| Data Analytics               | Dr. Neha Sharma   |
| Web Development              | Prof. Amit Verma  |
| Machine Learning             | Dr. Priya Mehta   |
| Python Programming           | Prof. Karan Singh |
+------------------------------+-------------------+
6 rows in set (0.01 sec)
```

Figure 16: Queries 6 – 7: students with departments, courses with instructors

```
mysql> select Student.Student_Name, Course.Course_Name, Enrollment.Status
    -> from Enrollment
    -> join Student
    -> on Enrollment.Student_ID = Student.Student_ID
    -> join Course
    -> on Enrollment.Course_ID = Course.Course_ID;
+---------------+------------------------------+------------+
| Student_Name  | Course_Name                  | Status     |
+---------------+------------------------------+------------+
| Rahul Sharma  | Database Management Systems  | Active     |
| Aman Kumar    | Database Management Systems  | Active     |
| Rohan Gupta   | Database Management Systems  | Active     |
| Priya Singh   | Data Analytics               | Active     |
| Simran Kaur   | Data Analytics               | Active     |
| Neha Verma    | Web Development              | Active     |
| Anjali Sharma | Web Development              | Active     |
| Priya Singh   | Machine Learning             | Active     |
| Arjun Mehta   | Machine Learning             | Active     |
| Rahul Sharma  | Python Programming           | Active     |
| Rohan Gupta   | Python Programming           | Active     |
| Anjali Sharma | Python Programming           | Completed  |
| Aman Kumar    | Advanced SQL                 | Active     |
+---------------+------------------------------+------------+
13 rows in set (0.00 sec)

mysql> select Module.Module_Name, Course.Course_Name
    -> from Module
    -> join Course
    -> on Module.Course_ID = Course.Course_ID;
+----------------------------------+------------------------------+
| Module_Name                      | Course_Name                  |
+----------------------------------+------------------------------+
| Introduction to DBMS             | Database Management Systems  |
| SQL Fundamentals                 | Database Management Systems  |
| Introduction to Data Analytics   | Data Analytics               |
| Data Visualization               | Data Analytics               |
| HTML Basics                      | Web Development              |
| CSS Fundamentals                 | Web Development              |
| Introduction to Machine Learning | Machine Learning             |
| Supervised Learning              | Machine Learning             |
| Python Basics                    | Python Programming           |
| Control Statements               | Python Programming           |
| Advanced Joins                   | Advanced SQL                 |
| Query Optimization               | Advanced SQL                 |
+----------------------------------+------------------------------+
12 rows in set (0.00 sec)
```

Figure 17: Queries 8 – 9: students with courses, modules with courses

```
mysql> select Student.Student_Name, Assessment.Assessment_Name, Grade.Marks_Obtained
    -> from Grade
    -> join Submission
    -> on Grade.Submission_ID = Submission.Submission_ID
    -> join Student
    -> on Submission.Student_ID = Student.Student_ID
    -> join Assessment
    -> on Submission.Assessment_ID = Assessment.Assessment_ID;
+---------------+------------------------------+----------------+
| Student_Name  | Assessment_Name              | Marks_Obtained |
+---------------+------------------------------+----------------+
| Rahul Sharma  | DBMS Quiz                    |          18.00 |
| Aman Kumar    | DBMS Quiz                    |          15.00 |
| Rahul Sharma  | SQL Assignment               |          44.00 |
| Priya Singh   | Analytics Quiz               |          17.00 |
| Simran Kaur   | Analytics Quiz               |          14.00 |
| Simran Kaur   | Data Analysis Project        |          42.00 |
| Neha Verma    | HTML CSS Test                |          21.00 |
| Priya Singh   | ML Midterm Exam              |          76.00 |
| Arjun Mehta   | ML Midterm Exam              |          85.00 |
| Rohan Gupta   | Python Programming Assignment |         46.00 |
| Anjali Sharma | Python Programming Assignment |         39.00 |
| Aman Kumar    | Advanced SQL Test            |          88.00 |
+---------------+------------------------------+----------------+
12 rows in set (0.00 sec)

mysql> select count(*) as Total_Students
    -> from Student;
+----------------+
| Total_Students |
+----------------+
|              8 |
+----------------+
1 row in set (0.01 sec)

mysql> select avg(Marks_Obtained) as Average_Marks
    -> from Grade;
+---------------+
| Average_Marks |
+---------------+
|     42.083333 |
+---------------+
1 row in set (0.01 sec)
```

Figure 18: Query 10: marks join; Queries 11 – 12: COUNT and AVG

```
mysql> select max(Marks_Obtained) as Highest_Marks,
    -> min(Marks_Obtained) as Lowest_Marks
    -> from Grade;
+---------------+--------------+
| Highest_Marks | Lowest_Marks |
+---------------+--------------+
|         88.00 |        14.00 |
+---------------+--------------+
1 row in set (0.01 sec)

mysql> select Course.Course_Name, count(Enrollment.Student_ID) as Total_Students
    -> from Course
    -> left join Enrollment
    -> on Course.Course_ID = Enrollment.Course_ID
    -> group by Course.Course_ID, Course.Course_Name;
+------------------------------+----------------+
| Course_Name                  | Total_Students |
+------------------------------+----------------+
| Database Management Systems  |              3 |
| Data Analytics               |              2 |
| Web Development              |              2 |
| Machine Learning             |              2 |
| Python Programming           |              3 |
| Advanced SQL                 |              1 |
+------------------------------+----------------+
6 rows in set (0.00 sec)

mysql> select Course.Course_Name, count(Enrollment.Student_ID) as Total_Students
    -> from Course
    -> join Enrollment
    -> on Course.Course_ID = Enrollment.Course_ID
    -> group by Course.Course_ID, Course.Course_Name
    -> having count(Enrollment.Student_ID) > 1;
+------------------------------+----------------+
| Course_Name                  | Total_Students |
+------------------------------+----------------+
| Database Management Systems  |              3 |
| Data Analytics               |              2 |
| Web Development              |              2 |
| Machine Learning             |              2 |
| Python Programming           |              3 |
+------------------------------+----------------+
5 rows in set (0.00 sec)
```

Figure 19: Queries 13 – 15: MAX/MIN, enrollments per course, HAVING

Output 7: GROUP BY, Subqueries and the IN Operator (Queries 16 – 20)

- **Query 16** gave assessment-wise averages: DBMS Quiz 16.5, SQL Assignment 44.0, Analytics Quiz 15.5, Data Analysis Project 42.0, HTML CSS Test 21.0, ML Midterm Exam 80.5, Python Programming Assignment 42.5 and Advanced SQL Test 88.0.
- **Query 17** returned 5 students who scored above the overall average of 42.08: Rahul Sharma (44), Arjun Mehta (85), Rohan Gupta (46), Aman Kumar (88) and Priya Singh (76).
- **Query 18**: the average credit value is 3.5, so only the three 4-credit courses are returned.

- **Query 19** returned the two Data Science students, Priya Singh and Simran Kaur. **Query 20** returned all 8 students because each has at least one submission.

```
mysql> select Assessment.Assessment_Name,
    ->  avg(Grade.Marks_Obtained) as Average_Marks
    -> from Assessment
    -> join Submission
    -> on Assessment.Assessment_ID = Submission.Assessment_ID
    -> join Grade
    -> on Submission.Submission_ID = Grade.Submission_ID
    -> group by Assessment.Assessment_ID, Assessment.Assessment_Name;
+--------------------------------+---------------+
| Assessment_Name                | Average_Marks |
+--------------------------------+---------------+
| DBMS Quiz                      |     16.500000 |
| SQL Assignment                 |     44.000000 |
| Analytics Quiz                 |     15.500000 |
| Data Analysis Project          |     42.000000 |
| HTML CSS Test                  |     21.000000 |
| ML Midterm Exam                |     80.500000 |
| Python Programming Assignment  |     42.500000 |
| Advanced SQL Test              |     88.000000 |
+--------------------------------+---------------+
8 rows in set (0.01 sec)

mysql> select Student.Student_Name, Grade.Marks_Obtained
    -> from Student
    -> join Submission
    -> on Student.Student_ID = Submission.Student_ID
    -> join Grade
    -> on Submission.Submission_ID = Grade.Submission_ID
    -> where Grade.Marks_Obtained > (
    ->  select avg(Marks_Obtained)
    -> from Grade);
+--------------+----------------+
| Student_Name | Marks_Obtained |
+--------------+----------------+
| Rahul Sharma |          44.00 |
| Arjun Mehta  |          85.00 |
| Rohan Gupta  |          46.00 |
| Aman Kumar   |          88.00 |
| Priya Singh  |          76.00 |
+--------------+----------------+
5 rows in set (0.00 sec)
```

Figure 20: Query 16: assessment-wise average marks; Query 17: students above the overall average

```
mysql> select Course_Name, Credits
    -> from Course
    -> where Credits > (
    -> select avg(Credits)
    -> from Course);
+----------------------------+---------+
| Course_Name                | Credits |
+----------------------------+---------+
| Database Management Systems |       4 |
| Data Analytics             |       4 |
| Machine Learning           |       4 |
+----------------------------+---------+
3 rows in set (0.00 sec)

mysql> select Student_Name
    -> from Student
    -> where Department_ID in (
    -> select Department_ID
    ->  from Department
    ->  where Department_Name = 'Data Science');
+--------------+
| Student_Name |
+--------------+
| Priya Singh  |
| Simran Kaur  |
+--------------+
2 rows in set (0.00 sec)

mysql> select Student_Name
    -> from Student
    -> where Student_ID in (
    -> select Student_ID
    -> from Submission);
+---------------+
| Student_Name  |
+---------------+
| Rahul Sharma  |
| Priya Singh   |
| Aman Kumar    |
| Neha Verma    |
| Arjun Mehta   |
| Simran Kaur   |
| Rohan Gupta   |
| Anjali Sharma |
+---------------+
8 rows in set (0.00 sec)
```

Figure 21: Query 18: above-average credits; Queries 19 – 20: IN operator

Output 8: EXISTS, ANY, ALL, View and Index (Queries 21 – 26)

- **Query 21** (`EXISTS`) returned all 5 instructors, since each teaches at least one course.
- **Query 22** (`NOT EXISTS`) returned an *Empty set*. This is the expected result because every one of the 8 students has submitted at least one assessment, which matches Query 20.
- **Queries 23 and 24** (`ANY` and `ALL`) also returned an *Empty set*. The only Data Science course (Data Analytics) has 4 credits, which is the highest credit value in the Course table, so no course has more than 4 credits.
- **Query 25**: `Student_Course_View` was created and returned all **13 enrollment** records with student and course names.
- **Query 26**: `Idx_Course_Name` was created, and the search for "Python Programming" returned the matching course row. `SHOW TABLES` now lists 11 objects (10 tables and 1 view).

```
mysql> select Instructor_Name
    -> from Instructor i
    -> where exists (
    -> select * from Course c
    -> where c.Instructor_ID = i.Instructor_ID);
+------------------+
| Instructor_Name  |
+------------------+
| Dr. Rajesh Kumar |
| Dr. Neha Sharma  |
| Prof. Amit Verma |
| Dr. Priya Mehta  |
| Prof. Karan Singh |
+------------------+
5 rows in set (0.00 sec)

mysql> select Student_Name
    -> from Student s
    -> where not exists (
    -> select * from Submission sub
    -> where sub.Student_ID = s.Student_ID);
Empty set (0.00 sec)

mysql> select Course_Name, Credits
    -> from Course
    -> where Credits > any (
    -> select Credits
    ->  from Course
    -> where Department_ID = 2);
Empty set (0.00 sec)
```

Figure 22: Queries 21 – 23: EXISTS, NOT EXISTS and ANY

```
mysql> select Course_Name, Credits
    -> from Course
    -> where Credits > all (
    -> select Credits
    -> from Course
    -> where Department_ID = 2);
Empty set (0.01 sec)

mysql> create view Student_Course_View as
    -> select Student.Student_ID,
    -> Student.Student_Name,
    -> Course.Course_Name,
    -> Enrollment.Enrollment_Date,
    -> Enrollment.Status
    -> from Student
    -> join Enrollment
    -> on Student.Student_ID = Enrollment.Student_ID
    -> join Course
    -> on Enrollment.Course_ID = Course.Course_ID;
Query OK, 0 rows affected (0.02 sec)

mysql> select * from Student_Course_View;
+------------+--------------+----------------------------+-----------------+-----------+
| Student_ID | Student_Name | Course_Name                | Enrollment_Date | Status    |
+------------+--------------+----------------------------+-----------------+-----------+
|        101 | Rahul Sharma | Database Management Systems | 2026-02-01      | Active    |
|        103 | Aman Kumar   | Database Management Systems | 2026-02-02      | Active    |
|        107 | Rohan Gupta  | Database Management Systems | 2026-02-06      | Active    |
|        102 | Priya Singh  | Data Analytics             | 2026-02-01      | Active    |
|        106 | Simran Kaur  | Data Analytics             | 2026-02-03      | Active    |
|        104 | Neha Verma   | Web Development            | 2026-02-01      | Active    |
|        108 | Anjali Sharma | Web Development           | 2026-02-05      | Active    |
|        102 | Priya Singh  | Machine Learning           | 2026-02-03      | Active    |
|        105 | Arjun Mehta  | Machine Learning           | 2026-02-05      | Active    |
|        101 | Rahul Sharma | Python Programming         | 2026-02-02      | Active    |
|        107 | Rohan Gupta  | Python Programming         | 2026-02-04      | Active    |
|        108 | Anjali Sharma | Python Programming        | 2026-02-07      | Completed |
|        103 | Aman Kumar   | Advanced SQL               | 2026-02-04      | Active    |
+------------+--------------+----------------------------+-----------------+-----------+
13 rows in set (0.01 sec)

mysql> create index Idx_Course_Name
    -> on Course(Course_Name);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Figure 23: Query 24: ALL; Query 25: view; Query 26: index

```
mysql> select *
    -> from Course
    -> where Course_Name = 'Python Programming';
+-----------+--------------------+-----------------------------+---------+---------------+---------------+
| Course_ID | Course_Name        | Course_Description          | Credits | Instructor_ID | Department_ID |
+-----------+--------------------+-----------------------------+---------+---------------+---------------+
|       305 | Python Programming | Python programming fundamentals |       3 |           205 |             1 |
+-----------+--------------------+-----------------------------+---------+---------------+---------------+
1 row in set (0.00 sec)

mysql> show tables;
+---------------------+
| Tables_in_lms       |
+---------------------+
| assessment          |
| course              |
| department          |
| enrollment          |
| grade               |
| instructor          |
| learning_resource   |
| module              |
| student             |
| student_course_view |
| submission          |
+---------------------+
11 rows in set (0.00 sec)

mysql> select * from Student_Course_View;
+------------+--------------+----------------------------+-----------------+-----------+
| Student_ID | Student_Name | Course_Name                | Enrollment_Date | Status    |
+------------+--------------+----------------------------+-----------------+-----------+
|        103 | Aman Kumar   | Advanced SQL               | 2026-02-04      | Active    |
|        102 | Priya Singh  | Data Analytics             | 2026-02-01      | Active    |
|        106 | Simran Kaur  | Data Analytics             | 2026-02-03      | Active    |
|        101 | Rahul Sharma | Database Management Systems | 2026-02-01      | Active    |
|        103 | Aman Kumar   | Database Management Systems | 2026-02-02      | Active    |
|        107 | Rohan Gupta  | Database Management Systems | 2026-02-06      | Active    |
|        102 | Priya Singh  | Machine Learning           | 2026-02-03      | Active    |
|        105 | Arjun Mehta  | Machine Learning           | 2026-02-05      | Active    |
|        101 | Rahul Sharma | Python Programming         | 2026-02-02      | Active    |
|        107 | Rohan Gupta  | Python Programming         | 2026-02-04      | Active    |
|        108 | Anjali Sharma | Python Programming        | 2026-02-07      | Completed |
|        104 | Neha Verma   | Web Development            | 2026-02-01      | Active    |
|        108 | Anjali Sharma | Web Development           | 2026-02-05      | Active    |
+------------+--------------+----------------------------+-----------------+-----------+
13 rows in set (0.00 sec)
```

Figure 24: Testing the index with a search, listing tables (now including the view), and displaying the view

Output 9: Final Verification

- The final `SELECT` statements confirm that the Course table (6 rows) and the Grade table (12 rows) hold the expected data at the end of the session.

```
mysql> select * from Course;
+-----------+-----------------------------+-----------------------------------------------+---------+---------------+---------------+
| Course_ID | Course_Name                 | Course_Description                            | Credits | Instructor_ID | Department_ID |
+-----------+-----------------------------+-----------------------------------------------+---------+---------------+---------------+
|       301 | Database Management Systems | Introduction to DBMS and SQL                  |       4 |           201 |             1 |
|       302 | Data Analytics              | Data analysis and visualization               |       4 |           202 |             2 |
|       303 | Web Development             | HTML CSS and JavaScript                       |       3 |           203 |             3 |
|       304 | Machine Learning            | Machine learning algorithms and applications  |       4 |           204 |             4 |
|       305 | Python Programming          | Python programming fundamentals               |       3 |           205 |             1 |
|       306 | Advanced SQL                | Advanced database queries and optimization    |       3 |           201 |             1 |
+-----------+-----------------------------+-----------------------------------------------+---------+---------------+---------------+
6 rows in set (0.00 sec)

mysql> select * from Grade;
+----------+---------------+----------------+-------+--------------------------+
| Grade_ID | Submission_ID | Marks_Obtained | Grade | Feedback                 |
+----------+---------------+----------------+-------+--------------------------+
|      901 |           801 |          18.00 | A     | Excellent performance    |
|      902 |           802 |          44.00 | A     | Very good SQL skills     |
|      903 |           803 |          15.00 | B     | Good understanding       |
|      904 |           804 |          17.00 | A     | Well done                |
|      905 |           805 |          42.00 | A     | Good analytical work     |
|      906 |           806 |          21.00 | B     | Good effort              |
|      907 |           807 |          85.00 | A     | Excellent understanding  |
|      908 |           808 |          46.00 | A     | Very good Python skills  |
|      909 |           809 |          39.00 | B     | Good work                |
|      910 |           810 |          88.00 | A     | Excellent SQL knowledge  |
|      911 |           811 |          14.00 | B     | Satisfactory performance |
|      912 |           812 |          76.00 | B     | Good understanding       |
+----------+---------------+----------------+-------+--------------------------+
12 rows in set (0.00 sec)

mysql>
```

Figure 25: Final verification: contents of the Course and Grade tables

IX. Conclusion

The Online Learning Management System database was designed and implemented using MySQL 8.0. The project demonstrates how a relational database can organize educational information and maintain relationships among students, instructors, departments, courses, assessments, submissions, and grades.

The implementation covers fundamental and advanced SQL concepts, including primary keys, foreign keys, constraints, joins, aggregate functions, grouping, subqueries, views, and indexes. It provides a foundation for managing online learning activities and generating useful academic reports.

The results show 8 students enrolled across 6 courses with 13 enrollments, an average of 42.08 marks across the 12 graded submissions, and 5 students scoring above that average. The three queries that returned an empty set (NOT EXISTS, ANY and ALL) were correct for the sample data. Because the assessments have different maximum marks (20 to 100), the system can be extended in future by calculating percentages instead of raw marks, and by adding more records, user roles, and a front-end interface.

Through this project, the practical application of relational database concepts and SQL query techniques was demonstrated.

X. Learning Outcomes

After completing this project, the following learning outcomes were achieved:

1. Learned to design and implement a relational database using MySQL 8.0.
2. Understood the importance of primary keys, foreign keys, and database constraints.
3. Learned to establish relationships between multiple tables.
4. Practiced inserting multiple records using a single SQL statement.
5. Developed SQL queries using joins, aggregate functions, GROUP BY, and HAVING.
6. Understood the use of nested queries and IN, EXISTS, ANY, and ALL operators.
7. Learned to create views and indexes.
8. Improved the ability to retrieve and analyze related data from a structured database.
9. Understood the role of relational databases in real-world educational applications.